Field Hours

CSCE 662 Distributed Processing Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 90 minutes

Failures you are willing to admit

You start from the failure you are willing to admit. In this course a process crashes and stays down, and a message may be lost or delayed, but a live process does not send a forged message. That crash-stop model is why a timeout exists: after 3 missed heartbeats, 1 second apart, you declare the peer dead. You may be wrong if the peer is only slow, so every later protocol has to tolerate a false suspicion.

You name the message protocol before you name the service. A request carries a client id and a unique number, and the reply quotes that number, so you can match them. With 2 requests in flight you must not apply the reply for number 9 to the call that sent number 10. A protocol that only says send the bytes and hope cannot support the file service or the transaction later. You write the fields down.

Crash-stop is the model. A forged message from a live process is a different course.

Admitted

Crash, and stay down

Lose or delay a message

3 missed beats, then dead

Not in this model

A live process forges data

Clocks that all agree

A timeout that is knowledge

Three missed heartbeats

1. Heartbeats are 1 second apart. You miss beat 1, beat 2, and beat 3.

2. After the third miss you mark the peer dead and you stop sending it new work.

3. The peer may only be slow. Any service you build next must stay correct if that suspicion was early.

Problem 1. Heartbeats are 1 second apart and you declare death after 3 misses. How long is the silence?

Answer. 3 seconds. That is a suspicion, not proof. A slow peer can still be alive, so later protocols must tolerate a false death.

Problem 2. Replies for request numbers 9 and 10 are both in flight. Which reply may you apply to the

call that sent 10?

Answer. Only the reply that quotes 10. Applying the reply for 9 to call 10 mixes two calls. The unique number is the match.

Lecture 2 · 90 minutes

Remote calls and duplicate replies

You call a remote procedure through a stub that looks local and is not. The client stub packs arguments, sends the request, and waits. The server stub unpacks, calls the procedure, and sends the reply. A timeout of 2 seconds without a reply means you do not know whether the procedure ran. Retrying blindly can run a transfer twice. The mechanism that stops that is an at-most-once table: the server remembers the unique number and the reply it already computed.

You either make the operation safe to repeat or you keep the at-most-once table. Adding 1 to a counter is not safe to run 2 times. Writing the same 4 bytes at the same offset is safe, because the second write leaves the same bits. For the add, the server stores the reply by unique id and returns it on a duplicate, without adding again. A client timeout can still follow success, so at-most-once is not exactly-once.

The stub hides packing, not failure. The table is what makes a retry safe for an add.

Pack and send -> Server runs or replays -> Reply quotes the id -> Timeout is not knowledge

Retry an increment

1. The client sends unique id 9 and asks the server to add 1 to a counter that is currently 5.

2. The server adds, stores reply 6 under id 9, and the reply is lost. After 2 seconds the client resends id 9.

3. The table returns 6 and does not add again. The counter stays 6. If the table had been missing, the counter would be 7.

Problem 1. A duplicate of unique id 9 asks the server to add 1 again. The stored reply is 6. What does the server do?

Answer. It returns 6 and does not add. The at-most-once table treats id 9 as already done. Running the add a second time would make the counter 7.

Problem 2. Writing the same 4 bytes to the same offset twice is safe, and adding 1 twice is not. Why the difference?

Answer. The second write of those 4 bytes leaves the same bits as the first. The second add changes the counter again. Idempotent writes can be retried; the add needs the reply table.

Lecture 3 · 90 minutes

Chunks, replicas, and leases

You split a file into chunks instead of locking the whole file. Each chunk is 64 megabytes, stored as 3 replicas. A write goes to the primary, which orders it and pushes the bytes to the other 2. A read may use any replica that has

applied that order. If you read one that is 1 update behind, you see stale bytes. That staleness is the consistency rule you chose, not a surprise.

You cache a chunk on the client with a lease. The lease is a time, here 10 seconds. While it holds, the client may read its copy and the primary recalls it before a conflicting write. When the lease ends, you drop the cache or renew. A cache with no lease can serve a value the other 2 replicas have already replaced. The directory maps the file name to the chunk ids and to the 3 servers that hold them.

The directory names holders. A lease is the only reason a client copy is still valid.

Client cache, leased 10 seconds

Directory of chunk ids

Primary plus 2 replicas

64 megabytes in one chunk

A write, then a stale read

1. The chunk is 64 megabytes and lives on 3 servers. The primary orders a write and pushes it to the other 2.

2. One replica is still 1 update behind. A read that lands there returns the old bytes.

3. A client cache without a lease of 10 seconds can do the same thing after the primary has moved on. With the lease, the primary recalls it before the conflicting write.

Problem 1. How many replicas hold each 64-megabyte chunk, and what do you see if you read one that is 1 update behind?

Answer. 3 replicas. You see the bytes from before that update. The protocol allows that staleness until the replica applies the primary's order.

Problem 2. A client cached a chunk and the lease was 10 seconds. The lease has ended and was not renewed. May the client keep serving the cache?

Answer. No. When the lease ends you drop the cache or renew it. Without a live lease the other replicas may already hold a newer write.

Lecture 4 · 90 minutes

Backups and checkpoints

You survive a crash by keeping a backup with the same state. The primary logs each update, waits until 1 backup has stored it, then replies. With that pair you tolerate 1 crash. If you reply before the log is stored, a dead primary loses an update the client thinks succeeded. On handover the backup becomes primary and applies any logged update that was not finished.

You checkpoint so a crash does not redo an hour of work. Every 10 minutes you store a snapshot plus the message log since then. Recovery loads the snapshot and replays the log. If the snapshot omits the receive buffer, you may replay a message already handled, so you include that buffer. Coordinated checkpointing pauses 2 processes, writes both snapshots, and resumes. Two uncoordinated snapshots can describe a state that never

existed.

Reply after the log is stored. A checkpoint that drops the receive buffer will replay a message.

Primary and backup

Log, then reply

Survive 1 crash

Backup becomes primary

Checkpoint

Snapshot every 10 minutes

Plus the log since then

Include the receive buffer

Reply too early

1. The primary writes the update into its log and replies to the client before the backup stores that log.

2. The primary crashes. The client already believes the update stuck. The backup never saw it.

3. Handover promotes the backup, which does not have the update. The correct order was: backup stores the log, then the primary replies.

Problem 1. You have 1 primary and 1 backup, and the backup has stored the log before you reply. How many crashes of those two can you tolerate without losing the acknowledged update?

Answer. 1. The surviving process has the log. Replying before the backup stores it means a crash of the primary can lose an update the client was told succeeded.

Problem 2. Two processes take snapshots at different times, with no pause. What can go wrong at recovery?

Answer. The two snapshots can describe a combined state that never existed. Coordinated checkpointing pauses both processes, writes both snapshots, then resumes. Also include each receive buffer or you may replay a handled message.

Lecture 5 · 90 minutes

Moving work without moving everything

You balance load by moving work, not by hoping equal connections mean equal cost. A least-connections dispatcher sends the next request to the server with the fewest open calls. If one of 4 servers holds 10 calls and the others hold 2, the next call goes to a server at 2. That rule fails when one call is a 30-second report and another is a 1-millisecond ping. You then weight by measured service time.

You place sessions with consistent hashing so one death does not move every key. Hash the key onto a ring. Each of 4 servers owns the arc ahead of its position. When 1 server leaves, only keys on its arc move, about a quarter, not the rest. Key mod 4 is the contrasting map: going from 4 servers to 3 remaps most keys, about three

quarters. You pick the ring when reshuffling the cache dominates the cost.

Least connections ignores service time. The ring moves one arc, not three quarters of the keys.

1. Count the open calls

2. Send to the short queue

3. Hash a key on the ring

4. Move only the dead arc

Four servers, one departure

1. Four servers sit on the ring. A new session hashes onto one arc. Least-connections is a different dispatcher: a server with 10 calls loses to one with 2.

2. One server dies. Only the keys on its arc move, about a quarter of them.

3. Key mod 4, reduced to 3 servers, remaps about three quarters of the keys. Those cache misses are the cost you avoided with the ring.

Problem 1. Servers hold 10, 2, 2, and 2 open calls. Least-connections routing: where does the next call go?

Answer. To a server that holds 2, not to the server that holds 10. If one of those short calls is a 30-second report, the count is the wrong weight and you should use measured service time.

Problem 2. You place keys with key mod 4, then a server dies and 3 remain. About what fraction of keys move, compared with one death on a 4-server ring?

Answer. Key mod 4 remaps about three quarters of the keys. One death on the ring moves about a quarter, the arc that server owned, and leaves the other arcs put.

Lecture 6 · 90 minutes

Two databases, one commit

You commit a transaction that touches 2 databases with two-phase commit. The coordinator sends prepare to both. Each participant writes a prepared record in its log and votes yes or no. If both vote yes, the coordinator writes commit and tells them. If 1 votes no, the coordinator writes abort. After a participant is prepared it cannot undo on its own, because the coordinator may already have told the other database to commit.

You pay for atomicity with blocking. If the coordinator dies after both of 2 participants are prepared and before they hear the decision, they wait. They cannot commit or abort until that decision returns. A timeout does not free them, because the other site may have committed. You log the decision before you send it, so a restarted coordinator can answer the 2 waiters.

Prepared means the participant has given up the right to choose. The coordinator's log is the choice.

Prepare both sites -> Collect yes or no -> Log commit or abort -> Tell the 2 sites

One no vote

1. The coordinator asks 2 databases to prepare. The first votes yes and writes a prepared record. The second votes no.

2. The coordinator writes abort, not commit, and tells both. The first database undoes.

3. If both had voted yes and the coordinator then died before sending the decision, both would wait. A timeout would not let either choose.

Problem 1. Both participants have voted yes and written prepared records. The coordinator crashes before sending a decision. What may each participant do?

Answer. Wait. Neither may commit or abort on its own, because the other site might already have been told the opposite. A timeout does not unlock them. A restarted coordinator answers from the decision it logged before sending.

Problem 2. One of 2 participants votes no. What does the coordinator write?

Answer. Abort. Commit requires both yes votes. The participant that voted yes must undo after it hears abort.

Lecture 7 · 90 minutes

Tickets and access

You separate authentication from authorization on every request. Authentication answers who sent it. A shared password sent on each call is a bad mechanism: it repeats a secret. You use a ticket a key server minted, carrying the client id, the server id, and an expiry of 8 hours, encrypted so only the file server can read it. The client presents that ticket instead of the password. A ticket copied after expiry fails the time check.

You authorize only after the ticket names a client. An access list says which ids may read and which may write. A capability is the other mechanism: holding an unforgeable token for the chunk is the right, so you check the token, not a list. With 3 replicas every replica enforces the same rule, or a client denied on the primary reads a backup. Encrypting bytes hides them on the wire. It does not decide who may ask.

The ticket names the client. The list or the capability says what that client may do.

Authentication

Ticket, not the password

Client id and server id

Expires in 8 hours

Authorization

Access list of ids

Or a capability token

Same rule on 3 replicas

Denied on the primary only

1. The ticket names the client and has 6 hours left on an 8-hour expiry, so the time check passes.

2. The primary's access list denies the write. Replica 2 of 3 does not check and serves the bytes.

3. That is a hole. All 3 replicas must enforce the same rule. Encrypting the bytes on the wire would not have closed it.

Problem 1. A ticket expired after 8 hours. The bytes on the wire are encrypted. May the file server still honor the ticket?

Answer. No. Expiry fails the time check. Encryption hides bytes from the network and does not extend the ticket.

Problem 2. The primary denies a client and one of the other 2 replicas does not check the access list. What can the client do?

Answer. Read the backup that skipped the check. With 3 replicas, every replica has to enforce the same list or the same capability. A check on the primary alone is not authorization.

Lecture 8 · 90 minutes

A request you can trace

You can now trace a request from a stub through a durable write. You attach a unique number so a retry runs at most once. You store the bytes as 3 replicas of one chunk and log them on a backup before you answer. You can fold that write into two-phase commit with a second database, and you can explain a block if the coordinator dies. You can also name who was allowed to call.

You can refuse a design that mixes these up. A load balancer that rewrites key mod 4 when a server dies will move about three quarters of the cached chunks, which is a different cost from losing 1 replica of 3. A ticket of 8 hours does not make the write atomic. You pick the mechanism that matches the failure you wrote down in the first lecture, and you can point at the log record that makes the claim true.

Each layer has a record you can point at. A timeout is not one of those records.

Unique id, at most once

Three replicas of the chunk

Log before the reply

Ticket, then the access rule

Early reply, and a prepared peer

1. The client writes 4 bytes. The primary logs them locally and replies before the backup stores the log, then the primary crashes.

2. The client was told the write succeeded. The backup may not have the 4 bytes, so handover can lose them.

3. A second database was already prepared in two-phase commit. It cannot commit or abort while the coordinator's decision is unknown. The early reply and the wait are two different failures.

Problem 1. A client writes 4 bytes. The primary replies before the backup stores the log, then crashes. A second database is already prepared for the same transaction. What was the client told, and what can the prepared database do?

Answer. The client was told the write succeeded, but the backup may lack the 4 bytes, so handover can lose them. The prepared database cannot commit or abort by itself while the decision is unknown. The early reply broke the backup rule, and two-phase commit still blocks the other site.

Problem 2. One of 3 replicas dies, and separately you rehash keys with key mod 4 down to 3 servers. Which event moves about three quarters of the cached keys?

Answer. The rehash from 4 servers to 3. Losing 1 replica of 3 drops a copy, and the other 2 still hold the chunk. Those are different costs. A ticket expiry of 8 hours moves neither.